Linux Learning Diary

Abstract

Author(s)	Publication type	Completion year
Last： Lin, First = Tong	Thesis, UAS	2026
	Number of pages 17	

Title of the thesis: Learning and Applying Linux Command-Line Tools for Efficient Data Management in CSC Computing Environments

Subheading: A Practical Exploration of Bash Commands, Scripting Techniques, and Data Processing Utilities

Degree

Engineer (UAS)

Name, title and organisation of the client

e.g. Clint Client, Design Engineer M.Sc. (Tech.), Design Agency Ltd.

Abstract

This thesis documents my learning process and practical experience with Linux command-line tools in the CSC computing environment. Throughout the study, I explored essential commands for navigating the file system, managing files and directories, processing data, and automating tasks using bash scripts. I also examined tools for searching files, checking file types, comparing data, verifying data integrity, and handling file permissions. In addition, I learned how to use packing and compression utilities such as tar, gzip, bzip2, zip, 7zip, and Zstandard to efficiently store and transfer large datasets. The thesis further covers data processing tools including grep, sed, awk, sort, uniq, and tr, which are widely used for filtering, transforming, and analyzing text-based data. By integrating these tools and scripting techniques, I gained a deeper understanding of how to work efficiently in Linux-based high-performance computing environments. This learning experience provides a foundation for future work involving automation, data analysis, and large-scale computational workflows.

Keywords

Linux, Bash, CSC, Command-Line Tools, File Management, Data Processing, Scripting, grep, sed, awk, tar, gzip, Compression, Automation, HPC

Table of Contents

Using Linux in command-line 4
Editing text and viewing images 6
Working with bash shell 8
More commands for managing files 10
Commands for data processing 13
Packing and compression tools 15
Linux bash scripts 17

Using Linux in command-line

I learned the fundamentals of using Linux from the command line, especially how to navigate directories, manage files, and use common commands. Before this, I mostly relied on graphical interfaces, so working directly in the terminal felt unfamiliar. But as I practiced the commands, I started to understand how efficient and powerful the command line can be.

One of the first things I learned is how the Linux file system is organized. Everything starts from the root directory /, and all files and folders are arranged in a hierarchical structure. The document explains that “the file systems in Linux machines are based on a hierarchical directory tree” . This helped me understand why every file has a unique path and why the current working directory matters.

Directory Navigation

I practiced several commands for moving around the directory tree:
pwd — prints the current working directory.
ls — lists files in the directory.
ls -la — shows detailed information, including permissions and hidden files. The document notes that the -l option “contains also information about the access permissions, the size and the modification time” .
cd directory_name — moves into a directory.
cd — returns to my home directory.
cd .. — moves one level up.
mkdir project1 — creates a new directory.
rmdir project1 — removes an empty directory.
Understanding the special directories . (current directory) and .. (parent directory) made navigation much easier.

Working With Files

I also learned how Linux treats files. Text files contain readable ASCII characters, while binary files contain non-text data. The document reminded me that Linux is case-sensitive, so filenames like Test.txt and test.txt are considered different.
I practiced several file-related commands:
cat filename — prints the whole file to the screen.
less filename — lets me scroll through the file one page at a time.
head filename — shows the first 10 lines.
tail filename — shows the last 10 lines.
tail -30 run1.log — shows the last 30 lines.
cp file1 file2 — copies a file.
cp file1 file2 results/ — copies files into a directory.
mv file1 file2 — renames or moves a file.
rm filename — deletes a file immediately.
rm -i filename — asks for confirmation before deleting.
I also learned that many users set an alias in .bashrc to make deletion safer:
alias rm="rm -i"

Wildcards and Special Characters

I practiced using wildcard characters:

? : matches exactly one character

* : matches any number of characters

For example:

ls test?.input :lists files like testA.input

ls test*.input: lists all files starting with test and ending with .input

I also learned the meaning of special characters:

> :redirects output to a file

>> : appends output

< :reads input from a file

|: pipes output to another command

For example:

ls test*.input > input_files :writes the list of matching files into a new file

ls -l | less: lets me scroll through a long directory listing

ls -l | grep "Aug 14" :filters files created on a specific date

These tools make it possible to combine simple commands into powerful workflows.

Editing text and viewing images

In this section, I am focusing on how to edit text files and how to view image or PostScript files in the CSC environment. This part was especially useful because editing configuration files and checking output images are common tasks when working on remote servers.

Text Editors in Linux

I learned that Linux provides several text editors, each with different levels of complexity. The document mentions that “editing text files… can be done with so-called text editor programs” , and that editors are mainly used for small files like scripts or parameter files.

1. nano

nano is the simplest editor and feels very beginner-friendly. I started it using:

nano file_name

Once inside nano, I practiced the basic shortcuts:

Ctrl-x — exit

Ctrl-o — save

Ctrl-r — read another file into the current buffer

Ctrl-v / Ctrl-y — move forward/backward one page

Ctrl-w — search text

Ctrl-k — cut a line

Ctrl-u — paste

The document explains that nano shows its commands at the bottom and that the ^ symbol means the Ctrl key. It also states that “Ctrl-6 marks the beginning of the text to be copied” , which I didn’t know before.

2. Emacs

Emacs is much more powerful and can run either in terminal mode or with a graphical interface. I started it with:

emacs file_name

In graphical mode, I can use menus like File, Edit, Options, and Tools, which makes it easier for beginners. The document also notes that Emacs supports syntax highlighting and multiple buffers, which is helpful for programming.

If I'm using Emacs in terminal mode, I need shortcuts such as:

Ctrl-x Ctrl-c — exit

Ctrl-x Ctrl-s — save

I can see why Emacs is considered a very flexible editor.

3. Vim

Vim was the most challenging part of today's learning. It starts with:

vi file_name or vim file_name

Vim has two main modes: command mode and insert mode. The document explains that this is why beginners often get confused. To enter insert mode, I press i, and to return to command mode, I press Esc.

I practiced the essential commands:

:w — save

:q — quit

:q! — quit without saving

i — insert text

a — append text

The document clearly warns that :q! should be used carefully because it discards all changes.

Even though Vim is harder to learn, I can see why it's popular among programmers.

Viewing Images and PostScript Files

I also learned how to view image files on CSC servers. The document states that "eog is a simple image viewing program that can display common image formats like png, bmp, jpg and tiff" .

To view a JPG file: eog image_name.jpg

For PostScript files, I can use Evince: evince file_name.ps

I also learned how to convert PostScript files to PDF: ps2pdf file1.ps

This command produces file1.pdf, which is convenient when I need to download or share the output.

Working with bash shell

In this section, I continued learning about how to work more efficiently in the bash shell. This part was surprisingly practical because it showed me how to speed up my workflow and avoid unnecessary typing. The document mentions that bash includes several features that "make command-line usage easier and more effective", and I immediately saw why.

Command-Line Editing

I practiced moving the cursor within a command using the arrow keys, and I also learned several useful keyboard shortcuts:
Ctrl-a — move to the beginning of the line
Ctrl-e — move to the end
Ctrl-k — cut everything to the right of the cursor
Ctrl-y — paste what was cut
These shortcuts make editing long commands much faster. I also learned that Ctrl-d deletes the character under the cursor, which is different from Backspace.

Command History

bash automatically stores previously executed commands in a file called .bash_history. I can view them with: history
I practiced using the up and down arrow keys to scroll through old commands. This is extremely helpful when I need to repeat similar commands, because I can just recall an old one and edit it slightly.
I also learned how to search through command history using Ctrl-r. When I press it and start typing, bash shows matching commands instantly. The document explains that "the search results will update with each character pressed", which makes it feel very interactive.

Automatic Tab Completion

This was one of the most useful features I learned today. Pressing Tab makes bash try to complete the command or filename I'm typing. It saves a lot of time and prevents typos. The example in the document was very clear. Instead of typing: evince final_research_report_new.pdf
I only need to type: evi + Tab
and bash completes it to: evince
Then typing:evince f + Tab
expands to: evince final_research_report_
Finally, adding one more n and pressing Tab completes the full filename. This really showed me how powerful auto-completion is.

Running Programs in the Background

I also learned how to run commands in the background by adding & at the end:eog image1.jpg &
This lets the program run while I continue using the terminal. If I want to bring it back to the foreground, I can use:fg

I practiced stopping programs with Ctrl-c and suspending them with Ctrl-z. Suspended jobs can be resumed in the background with: bg

Managing Processes
To see which processes I'm running, I used: ps
The output shows the PID (process ID), which I need if I want to terminate a process: kill PID
For example: kill 2001
The document also warns that sometimes a normal kill won't work, and in those cases I can use the stronger version: kill -9 PID
This should be used carefully because it forces the process to stop immediately.

More commands for managing files

In this section, I will study more tools that are essential for working efficiently in Linux. This part covered searching for files, checking file types, comparing files, verifying data integrity, encrypting files, and managing access permissions. Even though these topics are quite different, they all feel very practical for real work on CSC systems.

Using find to Locate Files

I learned that the find command is extremely powerful for searching files across directories. The basic syntax is: find directory search_condition

The document explains that “the default action… is -print, which prints the path and name of the matching files” . I practiced searching by filename: find ./ -name dataset27.txt

This returned: ./dataset3/dataset27.txt

I also learned that when using wildcards, I must quote the pattern: find $HOME/ -name "*.tmp"

I found the date-based search conditions very useful. For example, to find files not accessed in 28 days: find /scratch/project_2001234 -mtime +28

And to find files modified within the last 24 hours: find ./ -mtime -1

These will be helpful when cleaning up storage.

Checking File Types with file

The file command analyzes a file and prints a one-line description of its type. The document notes that it “recognizes most common text file formats, compressed files and Linux executables” .

I tried: file ./*

The output showed various types like ASCII text, gzip compressed data, JPEG image data, and HTML documents. This is useful when I’m unsure what kind of file I’m dealing with.

Counting Rows and Characters with wc

I practiced using wc to count lines: wc -l file_name

And to count how many .dat files are in the directory: ls *.dat | wc -l

This is a simple but very handy tool.

Comparing Files with diff

I learned that diff compares two files line by line: diff file1 file2

This is especially useful when checking differences between two versions of a script or configuration file.

Verifying Data Integrity with Checksums

I studied how checksums ensure that files are not corrupted during transfer or storage. The document explains that an md5 checksum is “a text string 32 characters long” and that any change in the file produces a completely different checksum.

To generate a checksum:md5sum poster1.pdf

To save it: md5sum poster1.pdf > poster1.pdf.md5

To verify: md5sum -c poster1.pdf.md5

This returned: poster1.pdf: OK

This is very useful for scientific datasets.

Encrypting Files with GPG

I learned how to encrypt files using GPG. The basic command is: gpg -c file_name

The program asks for a passphrase twice. After encryption, I get a .gpg file: my_file.txt.gpg

To decrypt: gpg my_file.txt.gpg

The document warns that “if you forget the password… there is no one who can open the file”, so I need to be careful.

Managing Access Permissions

Finally, I studied how Linux handles file permissions. Each file has permissions for owner, group, and others, and each category has read (r), write (w), and execute (x) permissions.

Using ls -l, I saw examples like:

-rw------- (owner can read/write, others have no access)

-rwxr-x--- (owner can read/write/execute, group can read/execute)

To modify permissions, I used chmod. For example, to give group members read access:

chmod g+r Test.txt

To give everyone full access: chmod go+rwx Test.txt

To remove permissions: chmod go-rwx Test.txt

I also learned that changing directory permissions does not automatically change the permissions of files inside it unless I use the recursive option: chmod -R g+w project1

This section helped me understand how Linux keeps files secure in a multi-user environment.

Commands for data processing

After going through file permissions and encryption, I moved on to learning various Linux tools for data processing. This section was especially interesting because it showed how powerful simple command-line tools can be when working with large datasets. The document mentions that Linux provides many tools to “study, filter and modify data files”, and I can now see how these commands fit together in real workflows.

Using grep to Filter Rows

I learned that the grep command is used to select rows that match a given pattern. The basic syntax is: grep pattern target_file. For example, using grep Pekka authors.txt prints two rows because the string “Pekka” also appears inside “Pekkanen”. If I want to match the full name including the space, I need to use quotation marks, such as grep "Pekka Niemi" authors.txt. I also practiced inverse selection with grep -v, case-insensitive search with grep -i, and regular expressions like the dot (.) for any character, ^ for beginning of line, and $ for end of line. I also learned that grep can be used in pipelines, such as filtering ls -l output to find files from a specific year.

Selecting Rows by Number with sed

The sed command can be used to pick specific rows from a file. For example, sed -n "3"p authors.txt prints the third row. I also studied an example where sed is used inside a loop to print multiple rows. This showed me how sed can be integrated into scripts for automated processing.

Selecting Columns with cut

The cut command allows selecting specific character ranges or columns. For example, cut -c2-5 authors.txt prints characters 2 to 5 from each row. To select columns, I can use -f together with a delimiter. For example, cut -d " " -f1,3 authors.txt prints the first and third columns using space as the delimiter. This is a simple but effective tool for structured text.

Processing Columns with awk

I learned that awk is a very powerful tool for column-based processing. It treats input as fields separated by spaces or tabs. For example, awk '{print $1 $3}' authors.txt prints the first and third columns together. I also learned how to add formatting, such as awk '{print "Location: "$3"\t Name: "$1" "$2}'. Using printf makes the output even cleaner. I practiced using conditions, such as printing only rows where the third column equals “Tampere”. I also learned that awk can perform numerical operations. For example, summing the file sizes of all jpg files by piping ls -l *.jpg into awk and accumulating the values in the fifth column. The document explains that the END block runs only after all input lines are processed, which helped me understand how cumulative calculations work.

Sorting Data with sort

The sort command can order rows alphabetically or numerically. For example, sort -k 2 authors.txt sorts by the second column. Numerical sorting can be done with sort -k4 -n authors.txt. I also learned how to sort using multiple keys, such as sort -k4n -k2 authors.txt, which sorts by the fourth column numerically and then by the second column alphabetically. The sorted output can be saved using either -o or redirection.

Removing Duplicate Rows with uniq

uniq removes successive duplicate rows, so sorting is usually needed first. For example, sort names | uniq removes duplicates correctly. Using uniq -c shows how many times each row appears. I also learned that uniq is case-sensitive unless I use the -i option.

Replacing Characters with tr

The tr command translates characters one by one. For example, tr "." "," < authors.txt replaces dots with commas. Using tr -d "." removes dots entirely. I also learned that tr works on characters, not words, so replacing “Oulu” with “Turku” would incorrectly convert characters one by one. I practiced converting lowercase to uppercase using tr "[:lower:]" "[:upper:]". This tool is simple but very useful for quick transformations.

Replacing Strings with sed

Finally, I learned how sed can replace strings using the syntax sed -e s/"search"/"replace"/g. For example, replacing “Oulu” with “Turku” in authors.txt updates all matching occurrences. I also practiced using regular expressions with sed, such as using $ to match the end of a line, . to match any character, and ^ to match the beginning of a line. These examples showed me how powerful sed can be for automated text editing, even though it takes time to master.

Packing and compression tools

In this section, I continued learning about packing and compression tools in Linux. These tools are essential when working with large datasets, especially in CSC environments where data often needs to be stored efficiently or transferred between systems. The document explains that archiving “packs multiple files into a single file” and compression “reduces its size without losing any data”, which makes both processes extremely useful in real workflows.

Using tar to Pack Files

I learned that tar is used to collect many files into one archive file, often called a tarball. By default, tar does not compress data; it only packs files together. The syntax follows the pattern: tar options tar_archive file. For example, archiving a directory like project_3 creates a file project_3.tar that contains all files inside the directory. I also learned that tar does not remove the original files, which makes it safe for backups. Extracting files is done with the x option, and I need to be careful because tar will overwrite existing files without warning. I also studied the commonly used options such as c for create, t for listing contents, x for extract, and f for specifying the archive file. Using z or j allows tar to compress the archive on the fly using gzip or bzip2. For example, creating project_3.tar.gz significantly reduces the file size compared to the uncompressed tar file.

Compressing Files with gzip

gzip is one of the most commonly used compression tools. It compresses a file and replaces it with a new file ending in .gz. The original file is removed unless I use redirection. Decompression is done with gunzip, which removes the .gz file and restores the original. I studied an example where a 1.5 GB file was compressed to 833 MB, showing how effective gzip can be. I also learned that gunzip is essentially gzip -d, so they share the same manual page.

Using bzip2 and bunzip2

bzip2 works similarly to gzip but uses a different algorithm that usually produces better compression, although it takes more time. The basic usage is bzip2 file_name, which creates a .bz2 file and removes the original. bunzip2 decompresses the file. I also learned about parallel versions pbzip2 and pbunzip2, which allow using multiple CPU cores with the -p option. This can significantly speed up compression for large files.

Using zip and unzip

zip is both an archiving and compression tool, unlike tar which only packs files. The syntax is zip -options archive_file source_name. I learned that zip does not include subdirectories unless I use the -r option. unzip can extract files, list contents, or extract only specific files. I studied an example where project_3.zip was created and later updated by adding sample10.txt. unzip -l shows the contents of the archive, and unzip archive file_name extracts only one file.

Using 7zip (7z)

7zip is widely used on Windows but also available on Linux. Before using it on Puhti, I need to load the module with module load p7zip. The basic syntax is 7z command -options archive_file file_names. The most important commands are a for adding files, e for extracting, and l for listing. I learned how to create a 7z archive of project_3 and how to add new files to an existing archive. Extracting individual files requires specifying the full path inside the archive, and extracted files appear in the current directory unless I use the -o option to define an output folder.

Using Zstandard (zstd)

Zstandard is a newer and very fast compression tool. Compressing a file is done with zstd file_name, which creates a .zst file. I can speed up compression by using multiple threads with the -T option, although on login nodes I should stick to one thread. Decompression is done with zstd -d file_name.zst. The document shows that Zstandard is significantly faster than older tools like gzip or zip, especially when using multiple cores.

Linux bash scripts

In this section, I learn how to automate tasks in Linux using bash scripts. Compared to full programming languages, bash is simpler and less powerful, but the document points out that “handy Linux scripts can be constructed in a few minutes”, and I can already see how useful they are for repetitive tasks in CSC environments.

Constructing a Script File

I learned that a script is just a normal text file created with editors like nano, Emacs, or vi. A script usually starts with the line #!/bin/bash, which tells the system to use the bash shell. After that, I can simply write the commands I want to automate. For example, creating a directory and copying all .map files into it can be done by writing mkdir mapfiles and cp *.map mapfiles/ inside the script. I also learned that lines starting with # are comments, except for the first line. To run the script, I can use source script_name, or I can give execution permission with chmod u+x script_name and run it directly with ./script_name.

Using Variables and Arrays

Bash variables are easy to define using variable=value without spaces. They are recalled using $variable or ${variable}. I practiced defining string variables like name=Veikko and printing them with echo. For integers, bash supports arithmetic using the syntax ((expression)). I learned operators like +, -, *, /, %, and **. Bash also supports one-dimensional arrays. For example, array=(a b c) defines an array, and ${array[2]} prints the third element. I also learned about special array variables like $0, $1, $2, which represent command-line arguments passed to a script. This allows scripts to accept input parameters, making them more flexible.

Quotation Marks

I studied the three types of quotation marks in bash: Double quotes allow variable substitution, single quotes take everything literally, and backticks `` execute a command and replace the quotes with its output. For example, echo "value = $variable" prints the variable’s value, while echo 'value = $variable' prints the literal text. Quotation marks are also necessary when arguments contain spaces, such as grep "file size" files.txt.

Loops and Conditional Statements

I learned how loops make scripts much more powerful. A for loop iterates over a list of items. For example, looping through sample1.txt to sample3.txt and printing their names. I also practiced using $(ls directory) to automatically generate the list of files. Bash also supports numeric for loops using the syntax for ((variable=start; variable<=end; variable++)), which is useful for processing numbered files. A while loop continues running as long as a condition is true. I studied an example where a variable is incremented until it reaches a certain value. Conditional statements use the if…then…else structure. I learned that bash uses different operators for strings and integers, such as == for strings and -eq for integers. I also learned file-related conditions like [[-e filename]] to check if a file exists. One example showed how to loop through a list of filenames and check which ones exist in the current directory.

Printing Output with echo and printf

I already used echo many times, but I learned that printf is better for formatted output. printf uses format definitions like %s for strings, %i for integers, and %f for floating-point numbers. I practiced aligning columns using width specifiers, such as %4i or %10s, and

adding control characters like \n and \t. This is especially useful when printing structured data inside scripts.